Astra Divinity: Documentación de Arquitectura de Software

1. Filtro W5HH (Alcance del Proyecto)

Definición estricta de los límites y objetivos del Producto Mínimo Viable (MVP) para la plataforma e-commerce Astra Divinity.

Dimensión	Definición Técnica y de Negocio
Qué (What)	MVP de plataforma e-commerce con catálogo, carrito de compras, checkout transaccional y panel administrativo.
Por qué (Why)	Establecer el canal de ventas digital oficial para la marca, con control de inventario preciso y centralizado.
Quién (Who)	Usuarios Finales (clientes), Administradores (gestión interna) y Arquitecto Full-Stack.
Cuándo (When)	Ejecución en 4 Sprints incrementales: DB/Docker, Autenticación, Transacciones, Frontend.
Dónde (Where)	Aplicación web mobile-first alojada en la nube (AWS Free Tier / Neon Serverless).
Cómo (How)	Backend: Node.js, Express, TypeScript. DB: PostgreSQL + Sequelize. Frontend: React. Infra: Docker.
Cuánto (How much)	Costo inicial minimizado ($0). Alta exigencia

Dimensión	Definición Técnica y de Negocio
	en diseño transaccional (ACID).

2. Casos de Uso (Modelado de Comportamiento)

Interacciones lógicas entre los actores y el sistema.

Actor: Cliente

- **CU-01 Exploración del Catálogo:** Consulta general y filtrado de productos (GET).
- **CU-02 Autenticación:** Registro e inicio de sesión mediante JWT y Bcrypt.
- **CU-03 Gestión de Carrito:** Manipulación temporal de selección de artículos.
- **CU-04 Checkout:** Bloque transaccional ACID (lectura de carrito, verificación de stock, creación de orden, commit o rollback).

Actor: Administrador

- **CU-05 Gestión de Inventario:** Operaciones CRUD sobre productos (protegidas por Auth/Role Middlewares).
- **CU-06 Auditoría de Órdenes:** Lectura de transacciones confirmadas.

3. Protocolos de Seguridad

- **Infraestructura:** PostgreSQL en red interna aislada de Docker (app_network). Caddy como proxy inverso forzando HTTPS y renovando certificados SSL automáticamente.
- **Aplicación:** Uso de Helmet para cabeceras HTTP seguras, CORS estricto, Rate Limiting y validación de esquemas de entrada.
- **Autenticación:** Hashing de contraseñas con bcryptjs y gestión de sesiones con JSON Web Tokens (JWT).
- **Autorización:** Escalamiento de privilegios cero desde endpoints públicos. Administradores generados por Seeders internos.
- **Integridad de Datos:** ORM Sequelize para prevenir SQL Injections y transacciones ACID para evitar cobros sin stock.

4. Diagrama de Entidad-Relación (DER)

Estructura relacional de PostgreSQL, normalizada e incluyendo el Patrón de Lista de Adyacencia para jerarquías infinitas.

Tabla: Categorias (Taxonomía Jerárquica)

- id (PK, Integer)
- nombre (String, NOT NULL)
- categoria_padre_id (FK -> Categorias.id, NULLABLE) - *Relación recursiva para subcategorías.*

Tabla: Usuarios

- id (PK, UUID)
- nombre (String, NOT NULL)
- email (String, UNIQUE, NOT NULL)
- password_hash (String, NOT NULL)
- rol (Enum: 'cliente', 'admin' - Default: 'cliente')

Tabla: Productos

- id (PK, Integer)
- categoria_id (FK -> Categorias.id, NOT NULL)
- nombre (String, NOT NULL)
- descripcion (Text)
- precio (Decimal, NOT NULL)
- stock (Integer, NOT NULL)
- talle (String)
- imagen_url (String)

Tabla: Ordenes

- id (PK, UUID)
- usuario_id (FK -> Usuarios.id, NOT NULL)
- total_compra (Decimal, NOT NULL)
- estado (Enum: 'pendiente', 'pagada', 'cancelada')
- fecha_creacion (Timestamp)

Tabla Pivote: Detalle_Ordenes (Order Items)

- id (PK, Integer)
- orden_id (FK -> Ordenes.id, NOT NULL)
- producto_id (FK -> Productos.id, NOT NULL)
- cantidad (Integer, NOT NULL)
- precio_unitario (Decimal, NOT NULL) - *Almacena el precio histórico exacto para proteger transacciones pasadas.*